Security Review Report
NM-0591 - ENS V2



NETHERMIND
SECURITY

(June 1, 2026)

Contents

1 Executive Summary **2**

2 Audited Files **3**
2.1 contracts-v2 . . . 3
2.2 verifiable-factory . . . 4

3 Summary of Issues **5**

4 System Overview **6**
4.1 Enhanced Access Control . . . 6
4.1.1 Key Functions . . . 6
4.2 ERC1155 Singleton Token . . . 6
4.2.1 Key Functions . . . 7
4.3 Hidden Contract Accounts (HCA) . . . 7
4.3.1 Key Functions . . . 7
4.4 PermissionedRegistry . . . 7
4.4.1 Key Functions . . . 8
4.5 Registrar and Pricing . . . 8
4.5.1 ETHRegistrar . . . 8
4.5.2 BatchRegistrar . . . 8
4.5.3 Pricing Oracle (StandardRentPriceOracle) . . . 9
4.6 PermissionedResolver . . . 9
4.6.1 Key Functions . . . 9
4.7 Universal Resolver . . . 9
4.8 Migration (ENS V1 → V2) . . . 10
4.8.1 Key Functions . . . 10
4.9 DNS TLD Integration . . . 11
4.10 Reverse Registrar . . . 11
4.11 Verifiable Factory . . . 11
4.11.1 Key Functions . . . 12

5 Risk Rating Methodology **13**

6 Issues **14**
6.1 [Medium] `UUPSProxy` upgrade validation relies on attacker-controlled `canUpgradeFrom(...)` check, allowing `WrapperRegistry` implementation replacement that bypasses frozen fuse restrictions 14
6.2 [Low] Expired name tokens can be transferred via `safeTransferFrom(...)` because `_update(...)` checks raw storage ownership instead of the expiry-aware `ownerOf(...)` . . . 16
6.3 [Low] Flipped operands in `gt(...)` make the non-payable check in `initialize(...)` dead code . . . 17
6.4 [Low] Reentrancy in `_regenerate(...)` via `ERC1155 onERC1155Received(...)` callback allows token transfer during role operations and `tokenVersionId` inflation . . . 18
6.5 [Low] Wildcard name roles do not authorize `setText(...)` and `setAddr(...)` due to a missing `resource(0, 0)` check in `onlyPartRoles` . . . 19
6.6 [Info] Name-level admins can grant key-specific text and addr roles but cannot revoke them . . . 22
6.7 [Info] Out-of-bounds memory write in `ResolverProfileRewriterLib.replaceNode(...)` due to unvalidated ABI offset pointers can return incorrect resolution data . . . 23
6.8 [Info] The proxy `receive(...)` function prevents delegation of plain ETH transfers to the implementation . . . 24
6.9 [Info] `setName(...)` and `setNameForAddr(...)` do not advance `inceptionOf`, leaving prior signatures valid after a direct name change . . . 25
6.10 [Best Practices] Unused code . . . 26

7 Documentation Evaluation **27**

8 Test Suite Evaluation **28**
8.1 Access Control & Role Constants . . . 29
8.2 Core Registry & ERC1155 . . . 29
8.3 DNS Module . . . 30
8.4 Registrar & Pricing . . . 31
8.5 Resolver . . . 31
8.6 Migration . . . 31
8.7 Miscellaneous . . . 32

9 About Nethermind **33**

1 Executive Summary

This document presents the results of the security review conducted by Nethermind Security for ENS-V2 contracts.

The Ethereum Name Service (ENS) is the leading decentralized naming protocol on Ethereum, mapping human-readable names (e.g., alice.eth) to addresses, content hashes, and other records. ENS V2 is a major architectural redesign of the protocol that transitions the system from a flat, single-registry model to a hierarchical tree of registries, in which each name owns its own registry contract responsible for managing its direct subdomains. This design unlocks flexible ownership models while preserving backward compatibility with unmigrated ENS V1 names through a dedicated migration framework.

The codebase is organized into different modules: a custom role-based access control layer (EnhancedAccessControl), a gas-optimized ERC-1155 singleton NFT base, the hierarchical registry system (PermissionedRegistry, UserRegistry, WrapperRegistry), resolvers (including a UniversalResolverV2 entry point), the ETHRegistrar with its rent price oracle, V1→V2 migration controllers (LockedMigrationController, UnlockedMigrationController), DNS integration (DNSTLDResolver, DNS TXT parsing), L2 reverse registrars, and the Hierarchical Canonical Address (HCA) helper library. Key security-sensitive mechanisms include the canonical token-ID system (which regenerates token IDs on role changes and re-registration to prevent marketplace griefing), paired admin/base roles with permission inheritance from a root resource, and UUPS upgradeability on the universal resolver.

The audit comprises 4430 lines of Solidity code. **The audit was performed using** (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report 10 points of attention, where one is classified as `Medium`, four are classified as `Low` and five are classified as `Informational` or `Best Practices` severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the system overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

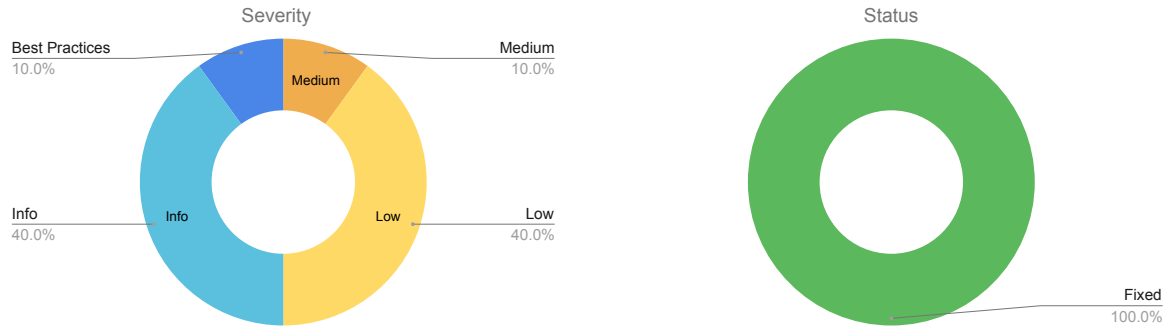


Fig. 1: Distribution of issues: Critical (0), **High** (0), **Medium** (1), **Low** (4), **Undetermined** (0), **Informational** (4), **Best Practices** (1). **Distribution of status: Fixed** (10), **Acknowledged** (0), **Mitigated** (0), **Unresolved** (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	April 17, 2026
Final Report	June 1, 2026
Initial Commit (contracts-v2)	e8696c6787e2e7a2fc17c67a3b06955a6b890adc
Initial Commit (verifiable-factory)	8361d8950ce690ae677ef11ca0cc21a4b9df34cc
Final Commit (contracts-v2)	5677359db15edd8b7e2a7cda4798d801ab129c9d
Final Commit (verifiable-factory)	04ec76f4c211bff795235feae90ad01882477ca1
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

2.1 contracts-v2

	Contract	LoC	Comments	Ratio	Blank	Total
1	contracts/src/CommonErrors.sol	3	8	266.7%	3	14
2	contracts/src/hca/HCAContext.sol	8	6	75.0%	3	17
3	contracts/src/hca/HCAContextUpgradeable.sol	8	4	50.0%	3	15
4	contracts/src/hca/HCAEquivalence.sol	14	23	164.3%	6	43
5	contracts/src/hca/interfaces/IHCAFactoryBasic.sol	4	6	150.0%	1	11
6	contracts/src/registrar/StandardRentPriceOracle.sol	195	141	72.3%	46	382
7	contracts/src/registrar/ETHRegistrar.sol	173	67	38.7%	33	273
8	contracts/src/registrar/BatchRegistrar.sol	31	19	61.3%	9	59
9	contracts/src/registrar/libraries/LibHalving.sol	83	24	28.9%	8	115
10	contracts/src/registrar/interfaces/IRentPriceOracle.sol	16	34	212.5%	11	61
11	contracts/src/registrar/interfaces/IETHRegistrar.sol	63	82	130.2%	22	167
12	contracts/src/utils/LibLabel.sol	9	7	77.8%	2	18
13	contracts/src/utils/LibString.sol	78	41	52.6%	17	136
14	contracts/src/utils/LibISO8601.sol	60	24	40.0%	13	97
15	contracts/src/utils/LibMem.sol	19	14	73.7%	5	38
16	contracts/src/utils/WrappedErrorLib.sol	37	15	40.5%	6	58
17	contracts/src/utils/interfaces/IUniversalSignatureValidator.sol	8	7	87.5%	1	16
18	contracts/src/reverse-registrar/L2ReverseRegistrarWithMigration.sol	23	24	104.3%	11	58
19	contracts/src/reverse-registrar/L2ReverseRegistrar.sol	200	131	65.5%	60	391
20	contracts/src/reverse-registrar/StandaloneReverseRegistrar.sol	80	77	96.2%	30	187
21	contracts/src/reverse-registrar/libraries/ChainIdsBuilderLib.sol	73	29	39.7%	11	113
22	contracts/src/reverse-registrar/interfaces/IL2ReverseRegistrar.sol	22	24	109.1%	7	53
23	contracts/src/reverse-registrar/interfaces/IContractName.sol	4	5	125.0%	1	10
24	contracts/src/registry/MetadataMixin.sol	14	16	114.3%	6	36
25	contracts/src/registry/PermissionedRegistry.sol	377	134	35.5%	49	560
26	contracts/src/registry/BaseUriRegistryMetadata.sol	28	28	100.0%	14	70
27	contracts/src/registry/WrapperRegistry.sol	121	40	33.1%	20	181
28	contracts/src/registry/SimpleRegistryMetadata.sol	29	28	96.6%	14	71
29	contracts/src/registry/UserRegistry.sol	31	24	77.4%	9	64
30	contracts/src/registry/libraries/RegistryRolesLib.sol	20	22	110.0%	9	51
31	contracts/src/registry/interfaces/IRegistryEvents.sol	33	38	115.2%	9	80
32	contracts/src/registry/interfaces/IPermissionedRegistry.sol	24	37	154.2%	15	76
33	contracts/src/registry/interfaces/IRegistryMetadata.sol	4	6	150.0%	1	11
34	contracts/src/registry/interfaces/IRegistry.sol	7	11	157.1%	4	22
35	contracts/src/registry/interfaces/IStandardRegistry.sol	24	46	191.7%	16	86
36	contracts/src/registry/interfaces/IWrapperRegistry.sol	14	11	78.6%	4	29
37	contracts/src/universalResolver/UpgradableUniversalResolverProxy.sol	87	56	64.4%	30	173
38	contracts/src/universalResolver/UniversalResolverV2.sol	32	31	96.9%	11	74
39	contracts/src/universalResolver/libraries/LibRegistry.sol	115	37	32.2%	9	161
40	contracts/src/access-control/EnhancedAccessControl.sol	259	166	64.1%	47	472
41	contracts/src/access-control/libraries/EACBaseRolesLib.sol	7	12	171.4%	2	21
42	contracts/src/access-control/interfaces/IEnhancedAccessControl.sol	44	75	170.5%	24	143
43	contracts/src/dns/DNSAliasResolver.sol	72	52	72.2%	14	138
44	contracts/src/dns/DNSTLDResolver.sol	269	119	44.2%	34	422
45	contracts/src/dns/DNSTXTResolver.sol	121	68	56.2%	14	203
46	contracts/src/dns/libraries/DNSTXTParserLib.sol	128	55	43.0%	5	188
47	contracts/src/migration/LockedMigrationController.sol	42	27	64.3%	12	81
48	contracts/src/migration/AbstractWrapperReceiver.sol	95	70	73.7%	19	184
49	contracts/src/migration/LockedWrapperReceiver.sol	129	55	42.6%	24	208
50	contracts/src/migration/UnlockedMigrationController.sol	82	50	61.0%	14	146
51	contracts/src/migration/libraries/LibMigration.sol	17	31	182.4%	12	60
52	contracts/src/resolver/ENSV1Resolver.sol	16	16	100.0%	8	40
53	contracts/src/resolver/PermissionedResolver.sol	446	185	41.5%	55	686
54	contracts/src/resolver/ENSV2Resolver.sol	25	18	72.0%	10	53
55	contracts/src/resolver/AbstractMirrorResolver.sol	40	23	57.5%	13	76
56	contracts/src/resolver/libraries/ResolverProfileRewriterLib.sol	26	17	65.4%	1	44
57	contracts/src/resolver/libraries/PermissionedResolverLib.sol	44	36	81.8%	13	93
58	contracts/src/resolver/interfaces/IPermissionedResolver.sol	17	31	182.4%	12	60
59	contracts/src/erc1155/ERC1155Singleton.sol	222	166	74.8%	41	429
60	contracts/src/erc1155/interfaces/IERC1155Singleton.sol	5	7	140.0%	2	14
	Total	**4277**	**2656**	**62.1%**	**905**	**7838**

2.2 verifiable-factory

	Contract	LoC	Comments	Ratio	Blank	Total
1	src/VerifiableFactory.sol	34	29	85.3%	12	75
2	src/IProxyAuthorization.sol	4	1	25.0%	1	6
3	src/UUPSProxy.sol	99	38	38.4%	28	165
4	src/IVerifiableFactory.sol	6	1	16.7%	3	10
5	src/IUUPSProxy.sol	10	1	10.0%	7	18
	Total	**153**	**70**	**45.8%**	**51**	**274**

3 Summary of Issues

	Finding	Severity	Update
1	`UUPSProxy` upgrade validation relies on attacker-controlled `canUpgradeFrom(...)` check, allowing `WrapperRegistry` implementation replacement that bypasses frozen fuse restrictions	Medium	Fixed
2	Expired name tokens can be transferred via `safeTransferFrom(...)` because `_update(...)` checks raw storage ownership instead of the expiry-aware `ownerOf(...)`	Low	Fixed
3	Flipped operands in `gt(...)` make the non-payable check in `initialize(...)` dead code	Low	Fixed
4	Reentrancy in `_regenerate(...)` via `ERC1155 onERC1155Received(...)` callback allows token transfer during role operations and `tokenVersionId` inflation	Low	Fixed
5	Wildcard name roles do not authorize `setText(...)` and `setAddr(...)` due to a missing `resource(0, 0)` check in `onlyPartRoles`	Low	Fixed
6	Name-level admins can grant key-specific text and addr roles but cannot revoke them	Info	Fixed
7	Out-of-bounds memory write in `ResolverProfileRewriterLib.replaceNode(...)` due to unvalidated ABI offset pointers can return incorrect resolution data	Info	Fixed
8	The proxy `receive(...)` function prevents delegation of plain ETH transfers to the implementation	Info	Fixed
9	`setName(...)` and `setNameForAddr(...)` do not advance `inceptionOf`, leaving prior signatures valid after a direct name change	Info	Fixed
10	Unused code	Best Practices	Fixed

4 System Overview

ENS V2 is a ground-up redesign of the Ethereum Name Service. It replaces the flat V1 registry with a **hierarchical, tokenized name registry**: every name can have its own registry contract managing its subdomains, and the ownership of a name is expressed as an **ERC1155 singleton token** minted by the owning registry. Permissions over a name are encoded as **bitmap-packed, resource-scoped role assignments** handled by a custom `EnhancedAccessControl` module, and the identity of the caller is resolved through an **HCA (Hidden Contract Accounts)** layer so that contract-based wallets are transparently attributed to their registered owners.

At a high level, resolving a name `sub.parent.eth` works by walking the registry tree from the **root registry** down one label at a time, stopping at the deepest registry that declares a resolver, and then delegating to that resolver via an ENSIP-10 `resolve()` call. Top-level `.eth` registrations are mediated by the `ETHRegistrar` (commit-reveal plus ERC20 payment into a `StandardRentPriceOracle`), while subname registration is performed directly on the owning registry by accounts holding the appropriate role (similar behavior to `.eth` registry can be achieved by granting the REGISTRAR role).

Names live under a three-state lifecycle:

- **`AVAILABLE`** — `expiry == 0` or `block.timestamp >= expiry`; no owner, no token.
- **`RESERVED`** — not expired and `owner == address(0)`; placeholder created by `BatchRegistrar` during V1 pre-migration, no token minted.
- **`REGISTERED`** — not expired and `owner != address(0)`; ERC1155 token minted, role bitmap granted.

Transitions `AVAILABLE → RESERVED/REGISTERED` require `ROLE_REGISTRAR` on `ROOT_RESOURCE`; `RESERVED → REGISTERED` requires `ROLE_REGISTER_RESERVED` on `ROOT_RESOURCE`; `REGISTERED → AVAILABLE` happens either via `unregister()` (which simply sets `expiry = block.timestamp`) or naturally by expiry.

The following subsections describe each module in the protocol, its responsibilities, and its most relevant entry points.

4.1 Enhanced Access Control

`EnhancedAccessControl` (EAC) is a custom RBAC system. Role assignments are **scoped to a resource**: each resource is an arbitrary `uint256` whose meaning is defined by the subclass (a name's canonical resource ID in `PermissionedRegistry`, a `keccak256(node, part)` hash in `PermissionedResolver`, etc.).

Roles are packed into a 256-bit bitmap with a specific nybble layout:

- **Bits 127–0** hold up to 32 **regular roles**, one 4-bit nybble each (only bit 0 of the nybble is meaningful for role membership; bits 1–3 are reserved).
- **Bits 255–128** hold the matching **admin roles**, at the same relative nybble position shifted by 128. Holding an admin role authorizes granting and revoking both the regular role and the admin role itself (`settable = adminBits » 128 | adminBits`).
- A special `ROOT_RESOURCE = 0` acts as a wildcard: any role granted at `ROOT_RESOURCE` is automatically satisfied for every other resource, because `hasRoles` combines the root bitmap with the resource bitmap via bitwise OR.

A parallel `_roleCount` bitmap uses the same nybble layout to track assignee counts per role, which imposes a hard cap of **15 assignees per role per resource** (4-bit counter). Subclasses hook into `_onRolesGranted` / `_onRolesRevoked` to react to role changes; `PermissionedRegistry` uses these callbacks to *regenerate* the ERC1155 token ID whenever permissions change. `grantRoles` and `revokeRoles` explicitly reject `ROOT_RESOURCE` and force callers through dedicated `grantRootRoles` / `revokeRootRoles` entry points.

4.1.1 Key Functions

- `grantRoles(resource, roleBitmap, account)` / `revokeRoles(...)` — grant or revoke a bitmap of roles on a specific (non-root) resource, gated by admin-role possession.
- `grantRootRoles(...)` / `revokeRootRoles(...)` — segregated path for root-level grants; `grantRoles`/`revokeRoles` revert when called with the root resource.
- `hasRoles(resource, roleBitmap, account)` — the universal permission check. Returns `true` if the bitmap is satisfied at the resource or at `ROOT_RESOURCE`.
- `_getSettableRoles(resource, account)` / `_getRevokableRoles(...)` — compute which roles an account may grant/revoke, combining its admin bitmap at the resource with its bitmap at root.
- `_transferRoles(from, to, resource)` — atomically moves all role assignments from one address to another within a resource; used by the registry on token transfer with callbacks disabled.

4.2 ERC1155 Singleton Token

Name ownership is represented by an **ERC1155 token** whose ID is a version-stamped labelhash. The system uses a custom `ERC1155Singleton` that enforces **exactly one owner per token ID**, replacing the standard `id → owner → balance` nested mapping with a flat `id → owner` mapping. `balanceOf` returns 1 if the account is the owner and 0 otherwise; any transfer of `value > 1` reverts with `ERC1155InsufficientBalance`; a transfer of `value = 0` is a no-op that still emits the standard event. An ERC721-style `ownerOf(id)` function is exposed on top of the ERC1155 interface (and the registry overrides it to add expiry and version validation).

The token system is built around a **versioned ID scheme**. For each name there are three IDs that all map to the same on-chain `Entry`:

ID	Formula	Purpose
`labelhash`	`keccak256(label)`	Canonical storage key
`tokenId`	`labelhash ⊕ uint32(labelhash) ⊕ tokenVersionId`	ERC1155 ownership
`resource`	`labelhash ⊕ uint32(labelhash) ⊕ eacVersionId`	EAC permission scope

`_entry(anyId)` zeroes the lower 32 bits (`LibLabel.withVersion(anyId, 0)`) to normalize any of these into the canonical storage slot. `eacVersionId` is bumped on unregister/re-register (so a re-registered name gets a **fresh permission scope**); `tokenVersionId` is bumped on unregister/re-register *and* on any **token regeneration** triggered by a role change.

4.2.1 Key Functions

- `_update(from, to, ids, values)` — enforces the single-owner invariant: for each id with `value > 0`, asserts `owners[id] == from` before writing `owners[id] = to` (or deleting on burn).
- `ownerOf(id)` — single-owner lookup; overridden by the registry to also enforce expiry and token-version validity.
- `safeTransferFrom(...)` — single-owner-enforcing ERC1155 transfer; the registry further requires `ROLE_CAN_TRANSFER_ADMIN` on the `from` address.

4.3 Hidden Contract Accounts (HCA)

The HCA layer provides a **sender-identity remapping** primitive. `HCAContext` (and its upgradeable variant `HCAContextUpgradeable`) override OpenZeppelin's `_msgSender()` so that whenever a transaction originates from a contract registered in the configured `HCA_FACTORY`, the protocol attributes the call to the HCA's **registered owner** rather than to the proxy contract itself. If `HCA_FACTORY` is unset or the caller is not a registered HCA, `msg.sender` passes through unchanged.

Because `EnhancedAccessControl` inherits `HCAContext`, **all role checks across the system use HCA-resolved identity**. This makes the HCA factory a system-wide trust anchor: a compromised HCA factory could return arbitrary "owner" addresses for any caller and enable impersonation of any account in the protocol. Every HCA-aware contract takes `HCA_FACTORY` as an immutable constructor argument.

4.3.1 Key Functions

- `HCAEquivalence._msgSenderWithHcaEquivalence()` — queries the factory; returns the account owner if `msg.sender` is a registered HCA, otherwise returns `msg.sender`.
- `HCAContext._msgSender()` / `HCAContextUpgradeable._msgSender()` — drop-in overrides inherited transparently by every authorization-sensitive contract in the system.

4.4 PermissionedRegistry

`PermissionedRegistry` is central to ENS V2: a tokenized (ERC1155) registry with resource-scoped access control that manages the set of subnames under a single parent node. Its storage centers on a per-label `Entry` struct:

```
struct Entry {
    uint32  eacVersionId;   // bumped on (un)register
    uint32  tokenVersionId; // bumped on (un)register and on regeneration
    IRegistry subregistry;  // child registry
    uint64  expiry;         // inclusive: block.timestamp >= expiry => AVAILABLE
    address resolver;       // resolver address
}
```

Many functions accept an `anyId` argument that can be a labelhash, `tokenId`, or EAC resource ID interchangeably, because `_entry()` normalizes by zeroing the lower 32 bits. The registry enforces the three-state name lifecycle (`AVAILABLE` / `RESERVED` / `REGISTERED`) with the following rules:

- **Self-call bypass.** `register()` skips the `ROLE_REGISTRAR` / `ROLE_REGISTER_RESERVED` check when `_msgSender() == address(this)`. This is used by `WrapperRegistry` during migration; `LockedMigrationController`, which calls the registry externally, must hold `ROLE_REGISTER_RESERVED` at `ROOT_RESOURCE`.
- **Transfer gating.** Actual transfers in `_update` additionally check `ROLE_CAN_TRANSFER_ADMIN` on the `from` address. Standard ERC1155 approval is *necessary but not sufficient*. On successful transfer, all per-token roles are transferred from `from` to `to` via `_transferRoles` with callbacks disabled.
- **Token regeneration.** Any `grantRoles`/`revokeRoles` on a registered token triggers `_onRolesGranted` / `_onRolesRevoked`, which bumps `tokenVersionId` and burns + mints the ERC1155 token so pending transfers against the stale `tokenId` revert.
- **Admin-role lockdown.** For registered token resources, `_getSettableRoles` returns `roleBitmap » 128` (admin bits stripped). Admin roles therefore cannot be granted on a token resource after registration; they can only be granted on `ROOT_RESOURCE` itself (via `grantRootRoles`), where they apply globally.
- **Expiry semantics.** Expiry is inclusive; expired entries return `address(0)` from `ownerOf`, `getResolver`, and `getSubregistry`, but the underlying storage is **not cleared** — it is masked by the expiry check.

Two UUPS-upgradeable specializations sit on top of `PermissionedRegistry`:

- **`UserRegistry`** — a minimal UUPS proxy pattern for user-managed subdomain registries; `_authorizeUpgrade` requires `ROLE_UPGRADE` at `ROOT_RESOURCE`.
- **`WrapperRegistry`** — one is deployed per migrated locked name by `LockedWrapperReceiver`. It doubles as the parent registry of the migrated name *and* as the migration receiver for its locked child tokens. Its public `register()` refuses to register labels that still correspond to unmigrated locked V1 children (`NameRequiresMigration`), and `getResolver()` returns the V1 resolver for still-migratable children so resolution is continuous across migration. `_inject()` calls `super.register()`, which bypasses both the migration guard and (via the self-call bypass) the `ROLE_REGISTER_RESERVED` check.

4.4.1 Key Functions

- `register(label, owner, subregistry, resolver, roleBitmap, expiry)` — encodes the full state machine: handles `AVAILABLE` → `RESERVED/REGISTERED` (requires `ROLE_REGISTRAR`) and `RESERVED` → `REGISTERED` (requires `ROLE_REGISTER_RESERVED`, with `expiry = 0` meaning "keep reserved expiry"), burns any previous token, bumps `eacVersionId/tokenVersionId` on re-registration, mints the token, and grants `roleBitmap`.
- `renew(tokenId, newExpiry)` — extends the expiry under `ROLE_RENEW`, enforcing that the new expiry is at least the current one.
- `unregister(tokenId)` — under `ROLE_UNREGISTER`, sets `expiry = block.timestamp`; if the entry is `REGISTERED`, also bumps both version counters and burns the token (does *not* clear `subregistry/resolver` storage).
- `setSubregistry` / `setResolver` — update the child registry or resolver on a token under `ROLE_SET_SUBREGISTRY` or `ROLE_SET_RESOLVER`.
- `safeTransferFrom(...)` — ERC1155 transfer; requires `ROLE_CAN_TRANSFER_ADMIN` on `from` and atomically moves all roles to `to`.
- `getStatus(anyId)` / `getState(anyId)` — read-only helpers used by the registrar and other consumers to determine whether a name is `AVAILABLE`, `RESERVED`, or `REGISTERED`, and to read its current entry.

4.5 Registrar and Pricing

Top-level `.eth` registration is handled by `ETHRegistrar`, paired with `StandardRentPriceOracle` for pricing. A lightweight `BatchRegistrar` exists for owner-only bulk *reservation* of V1 names ahead of migration.

4.5.1 ETHRegistrar

`ETHRegistrar` uses a two-step commit-reveal scheme to prevent front-running. All registration parameters (`label`, `owner`, `secret`, `subregistry`, `resolver`, `duration`, `referrer`) are bound into the commitment hash:

```
commitment = keccak256(abi.encode(
    label, owner, secret, subregistry, resolver, duration, referrer
));
```

`paymentToken` is **deliberately excluded** from the commitment so users can switch payment tokens at reveal time. A commitment must mature for at least `MIN_COMMITMENT_AGE` but not exceed `MAX_COMMITMENT_AGE`; successful reveal consumes and deletes the commitment (single-use). Registration mints the token with the fixed `REGISTRATION_ROLE_BITMAP`:

```
REGISTRATION_ROLE_BITMAP =
    ROLE_SET_SUBREGISTRY | ROLE_SET_SUBREGISTRY_ADMIN
  | ROLE_SET_RESOLVER    | ROLE_SET_RESOLVER_ADMIN
  | ROLE_CAN_TRANSFER_ADMIN;
```

This bitmap deliberately does **not** include `ROLE_RENEW` or `ROLE_UNREGISTER` — the registry-level renewal and unregistration paths must be mediated by the registrar itself (which holds the corresponding roles at `ROOT_RESOURCE`).

- `commit(commitment)` — records `commitmentAt[commitment] = block.timestamp`; refuses to overwrite an unexpired commitment.
- `register(label, owner, secret, subregistry, resolver, duration, paymentToken, referrer)` — validates the commitment window, checks availability, prices via the oracle, pulls ERC20 payment via `safeTransferFrom` to `BENEFICIARY`, and calls `PermissionedRegistry.register` with `REGISTRATION_ROLE_BITMAP`.
- `renew(label, duration, paymentToken)` — charges only the base rate (no premium on renewals) and extends the expiry via `PermissionedRegistry.renew`; uses `state.latestOwner` as the pricing context so prior owners never pay premium.
- `makeCommitment(...)` — canonical commitment hash construction helper.

4.5.2 BatchRegistrar

`BatchRegistrar` is an administrative, `Ownable`-gated tool used during V1 pre-migration. It calls `PermissionedRegistry.register(...)` directly with `owner = address(0)` and `roleBitmap = 0`, which creates `RESERVED` entries (no token minted, no payment collected). If a name is already `RESERVED` with a shorter expiry, it is renewed; `REGISTERED` names are skipped. The reservations it creates are later promoted to `REGISTERED` by the migration controllers.

4.5.3 Pricing Oracle (StandardRentPriceOracle)

The oracle implements the pricing model:

$$\text{Total} = \Big(\text{BaseUnits} - \text{DurationDiscount} + \text{ExpiryPremium}\Big) \times \text{ExchangeRate}$$

- **Base rate.** `baseRate(label)` looks up `_baseRatePerCp[min(codepoints, arrLen) - 1]`, so shorter names cost more per second.
- **Duration discount.** A piecewise-linear integrator `integratedDiscount(t + duration) - integratedDiscount(t)` over a configurable set of `DiscountPoint[]` waypoints rewards longer registrations.
- **Expiry premium.** An exponential (halving) decay via `LibHalving`: `halving(initial, half, elapsed) - halving(initial, half, premiumPeriod)`, so the premium is exactly zero at `elapsed == premiumPeriod`. The **prior owner** (as tracked by `state.latestOwner`) and the zero address pay *no* premium, which also allows price-quote queries against `address(0)`.
- **Exchange rate.** `Math.mulDiv` with ceiling rounding converts the internal base units into the chosen `paymentToken`; a payment token is accepted only if `_paymentRatios[token].denom > 0`.

4.6 PermissionedResolver

`PermissionedResolver` is the main record resolver for the protocol. It is UUPS-upgradeable, inherits `HCAContextUpgradeable`, and implements most ENSIP record profiles — `addr`, `addr(coinType)`, `text(key)`, `contenthash`, `name`, `pubkey`, `ABI`, `interfaceImplementer`, plus ENSIP-10 `resolve()` and `IMulticallable`. Records are held per-node under a `_versions[node]` counter, so `clearRecords(node)` simply bumps the version to logically wipe all records without touching storage.

Two features of this resolver are worth flagging:

- **Internal aliasing.** `setAlias(fromName, toName)` (root-only, via `ROLE_SET_ALIAS`) stores `_aliases[namehash(fromName)] = toName`. At resolution time, `getAlias()` performs longest-suffix matching by iterating over labels, applying prefix rewriting via `mcopy` assembly, and re-resolving until no further alias applies. Length-1 cycles are detected and applied once; length-2+ cycles are intentionally left to run out of gas.
- **4-dimensional permission model.** Resolver resources are `keccak256(node, part)` where `node = 0` and/or `part = 0` act as wildcards. The `onlyPartRoles` modifier checks, in order, `(node, part)`, `(0, part)`, `(node, 0)`, and falls back implicitly to `ROOT_RESOURCE` via `EnhancedAccessControl` — a total of 5 permission levels.

The generic `grantRoles` entry point is **disabled** on the resolver (it always reverts): role grants must go through the specialized `grantNameRoles`, `grantTextRoles`, `grantAddrRoles` functions, which grant on the correct part-scoped resource.

4.6.1 Key Functions

- `resolve(name, data)` — ENSIP-10 entry: resolves aliases, rewrites the node in the calldata via `replaceNode`, dispatches via `staticcall` to `this`. `multicall` payloads are decoded and each inner call dispatched individually.
- `setAlias(fromName, toName)` — root-only alias configuration.
- `setAddr`, `setText`, `setContenthash`, `setPubkey`, `setABI`, `setName`, `setInterface`, `clearRecords` — record setters, each gated by `onlyPartRoles` on the relevant role/part pair.
- `grantNameRoles` / `grantTextRoles` / `grantAddrRoles` — specialized role grants; the generic `grantRoles` is disabled.
- `multicall(bytes[])` / `multicallWithNodeCheck(node, bytes[])` — batched `delegatecall` entry point for record updates.

In addition to `PermissionedResolver`, the resolver layer includes **mirror resolvers** that delegate resolution to a different registry tree:

- `AbstractMirrorResolver` — base class that finds a target resolver via `_findResolver(name)` and dispatches via CCIP-Read-aware resolver invocation.
- `ENSV1Resolver` — uses the ENS V1 registry (`REGISTRY_V1`) to find resolvers, so V2 callers can resolve V1 names.
- `ENSV2Resolver` — uses `LibRegistry.findResolver` on the V2 root; optionally overrides the `.eth` node with a dedicated `ETH_RESOLVER`.

4.7 Universal Resolver

`UniversalResolverV2` is the public API that clients (wallets, dApps) call to resolve an arbitrary DNS-encoded name. It extends `AbstractUniversalResolver`, so CCIP-Read (EIP-3668) is supported out of the box. Its job is to **walk the V2 registry tree** starting from the immutable `ROOT_REGISTRY` trust anchor, and to locate the deepest registry on the path that declares a resolver — which then handles the ENSIP-10 `resolve` call.

Traversal logic lives in the `LibRegistry` helper library and exposes four distinct views:

- `findResolver(name)` — core traversal: returns `(resolver, node, offset)` for the deepest registry with a resolver set. The recursion depth equals the label count, so deeply nested names are a concern.
- `findRegistries(name)` — returns the full ancestry of registries in label order.

- `findExactRegistry(name)` — returns the registry owning exactly the given name, or zero if not canonical.
- `findCanonicalRegistry(name)` / `findCanonicalName(registry)` — round-tripping between a registry and its canonical DNS-encoded name, with consistency verification via `keccak256`.

`UniversalResolverV2` is deployed **behind** `UpgradableUniversalResolverProxy`, an EIP-1967-slot proxy that uses `staticcall` exclusively (the implementation is purely read-only) and performs `OffchainLookup` sender rewriting in its fallback so CCIP-Read errors are reported as originating from the proxy, not the implementation. The proxy admin can `upgradeTo(newImplementation)` or **permanently** `renounceAdmin()` (irreversible).

4.8 Migration (ENS V1 → V2)

ENS V2 preserves V1's existing name population through four distinct migration paths, all of which converge on the `.eth` `PermissionedRegistry`. Migration is **pull-based**: the user initiates it by transferring their V1 ownership token (a `NameWrapper` ERC1155 or a `BaseRegistrar` ERC721) to one of the migration receivers, which validates V1 state, translates it into V2 permissions, and promotes the pre-reserved V2 entry.

All paths depend on `BatchRegistrar` having previously created a `RESERVED` entry for the name, and they all pass `expiry = 0` to `PermissionedRegistry.register` as the `RESERVED → REGISTERED` promotion signal (which causes the registry to keep the reserved expiry).

The three **wrapped** paths share the same entry structure:

- The entry-point hook (`onERC1155Received/Batch`) validates `onlyWrapper` and the minimum `data` size.
- It then makes an external self-call to `finishERC1155Migration`, which asserts `msg.sender == address(this)` — the only way to reach `_migrateWrapped`.

The **Unwrapped ERC-721** path is different: `onERC721Received` validates `msg.sender == _REGISTRAR_V1` and the `data` size, then calls `_inject` directly — no self-call, no `_migrateWrapped`.

The four paths:

- **Locked 2LD** — `LockedMigrationController.onERC1155Received`. Validates that the V1 token has `CANNOT_UNWRAP` set (`NameNotLocked` otherwise), rejects frozen approvals (`FrozenTokenApproval`), clears or copies the V1 resolver depending on `CANNOT_SET_RESOLVER`, translates fuses to V2 roles via `_subregistryRoleBitmapFromFuses` and `_tokenRoleBitmapFromFuses`, deploys a dedicated per-name `WrapperRegistry` proxy via `VerifiableFactory.deployProxy` with `salt = uint256(node)`, and calls `ETH_REGISTRY.register(..., expiry = 0)`. This controller must hold `ROLE_REGISTER_RESERVED` at `ROOT_RESOURCE` on the `.eth` registry.
- **Locked subdomain (N+1 LD)** — on the parent's `WrapperRegistry.onERC1155Received`. The same `LockedWrapperReceiver` logic runs, but `_inject` uses `super.register()` which bypasses both the `NameRequiresMigration` guard and (via the self-call bypass because `_msgSender() == address(this)`) the `ROLE_REGISTER_RESERVED` check.
- **Unlocked wrapped** — `UnlockedMigrationController.onERC1155Received`. Rejects locked tokens, clears the V1 resolver via `NAME_WRAPPER.setResolver`, and injects with a fixed `REGISTRATION_ROLE_BITMAP` (the same bitmap as a fresh registration), since unlocked V1 names had no meaningful fuse restrictions worth preserving.
- **Unwrapped ERC-721** — `UnlockedMigrationController.onERC721Received`. Validates `msg.sender == _REGISTRAR_V1`, reclaims V1 ENS ownership via `BaseRegistrar.reclaim(tokenId, address(this))`, clears the V1 resolver directly via `_REGISTRY_V1.setResolver`, and injects with `REGISTRATION_ROLE_BITMAP`.

The locked-path **fuse-to-role translation** produces two bitmaps:

- **Subregistry roles** — `ROLE_REGISTRAR` unless `CANNOT_CREATE_SUBDOMAIN` is set, `ROLE_RENEW | ROLE_RENEW_ADMIN` always, and admin variants (`roleBitmap « 128`) only when the name is not frozen (`CANNOT_BURN_FUSES` not set).
- **Token roles** — `ROLE_RENEW` if `CAN_EXTEND_EXPIRY` is set, `ROLE_SET_RESOLVER` unless `CANNOT_SET_RESOLVER` is set, `ROLE_CAN_TRANSFER_ADMIN` unless `CANNOT_TRANSFER` is set, and admin variants only when not frozen.

The key invariant is that admin variants are only granted when `CANNOT_BURN_FUSES` is *not* burned — if the V1 name was frozen, the migrated V2 name cannot have its permissions expanded.

4.8.1 Key Functions

- `AbstractWrapperReceiver.onERC1155Received / onERC1155BatchReceived` — common entry point for every wrapped migration. Enforces `onlyWrapper` and minimum data size, then performs the wrapped self-call.
- `AbstractWrapperReceiver.finishERC1155Migration(...)` — self-call-only dispatcher that runs `_migrateWrapped`.
- `LockedWrapperReceiver._migrateWrapped(...)` — locked-path core: fuse validation, frozen-approval check, resolver handling, `WrapperRegistry` proxy deployment, fuse-to-role translation, and `_inject`.
- `LockedMigrationController._inject(...)` — external call to the `.eth` `PermissionedRegistry.register` for the locked 2LD path.
- `UnlockedMigrationController.onERC1155Received / onERC721Received / _inject` — unlocked wrapped and unwrapped paths; both use the fixed `REGISTRATION_ROLE_BITMAP`.
- `WrapperRegistry.initialize / register / getResolver / _inject` — per-name registry deployed for migrated locked 2LDs; handles the locked-child guard and V1-resolver continuity for unmigrated children.

4.9 DNS TLD Integration

The DNS module lets any DNSSEC-signed TLD (`.com`, `.io`, ...) be resolved through ENS V2 without the TLD being registered on-chain. It is built from three cooperating contracts plus a DFA-based TXT parser library:

- **`DNSTLDResolver`** — multi-step resolver. It first checks whether the V1 ENS registry already has a resolver for the name (and that it isn't the legacy DNS TLD resolver or `DNSTLDResolver` itself); if so, it delegates directly. Otherwise, it issues a CCIP-Read (EIP-3668) round-trip to fetch DNSSEC-verified TXT records from the configured oracle gateways. It verifies the DNSSEC proof against the `DNSSEC_ORACLE`, finds the first TXT record prefixed with `"ENS1 "`, and parses it into `(resolver, context)`. All functions are `view`; there is no access control — trust is delegated to the DNSSEC oracle and gateways. Immutables include `ENS_REGISTRY_V1`, `DNS_TLD_RESOLVER_V1`, `ROOT_REGISTRY`, `DNSSEC_ORACLE`, `ORACLE_GATEWAY_PROVIDER`, and `BATCH_GATEWAY_PROVIDER`.
- **`DNSTXTResolver`** — a stateless resolver that answers `addr`, `text`, `contenthash`, `pubkey`, and `multicall` profile queries by parsing a DNS TXT record `context` string on demand.
- **`DNSAliasResolver`** — a gasless resolver that rewrites the queried DNS name using an alias rule extracted from the TXT context (either suffix rewrite or full replace), then defers actual resolution to the V2 registry tree. It is intended to be called only by `DNSTLDResolver`; this convention is documentation-level and not enforced on-chain.
- **`DNSTXTParserLib`** — a deterministic finite-automaton parser for DNS TXT `key=value` pairs.

4.10 Reverse Registrar

The reverse registrar stores the **address → name** mapping used by UIs to display a human-readable primary name for an arbitrary account.

- **`StandaloneReverseRegistrar`** (abstract) — stores `_names: node → string`, implements ENSIP-10 wildcard resolution, and exposes `_setName(addr, name)` with *no* internal authorization: concrete subclasses must enforce access control around it.
- **`L2ReverseRegistrar`** — deployed to each L2 and concretely implements authorization. Its `CHAIN_ID` immutable is derived at construction from the ENSIP-11 coin type, and it delegates contract-signature verification to the Universal Signature Validator at the fixed mainnet address `0x164af34fAF9879394370C7f09064127C043A35E9` (supporting EIP-191 for EOAs, ERC-1271 for contracts, and ERC-6492 for counterfactual contracts).
- **`L2ReverseRegistrarWithMigration`** — adds an owner-gated `batchSetName(address[])` that reads names from the old V1 reverse registrar and copies them to the new one.

Authorization paths on `L2ReverseRegistrar`:

Method	Authorization
`setName(string)`	Self only (`_msgSender()`).
`setNameForAddr(address, string)`	Self, or `Ownable.owner()` of the target contract.
`setNameForAddrWithSignature(...)`	EIP-191/1271/6492 signature.
`setNameForOwnableWithSignature(...)`	Owner-signed claim for an Ownable contract.
`syncName(address)`	Anyone; reads from `IContractName.contractName()`.

4.11 Verifiable Factory

The `verifiable-factory` repository is the deployment substrate for ENS V2's per-name `WrapperRegistry` proxies. It does two things: (1) deploys UUPS proxies at **deterministic `CREATE2` addresses** derived from a salt, and (2) lets any on-chain consumer later **verify** that a given proxy was indeed produced by this factory, without trusting off-chain metadata. The contracts are stateless (no owner, no pausing) — any address can deploy proxies.

There are two main contracts:

- **`VerifiableFactory`** — stateless factory exposing `deployProxy(implementation, salt, data)` and `verifyContract(proxy, expectedImpl)`. Deployment computes `outerSalt = keccak256(msg.sender, salt)` so the caller is bound into the derived address, then deploys a `UUPSProxy` via `CREATE2` and calls `initialize(implementation, data)` on it in the same transaction.
- **`UUPSProxy`** — a custom (*not* OpenZeppelin) UUPS proxy that bakes `verifiableProxyFactory` and `_salt` into bytecode as `immutable`s. Its `fallback()` includes a **post-call implementation-change guard**: if a normal delegated call mutates the implementation slot, the call reverts. Only the explicit `upgradeToAndCall(newImpl, data)` path bypasses this guard.

Upgrades use a **dual-authorization model**: `upgradeToAndCall` first calls `newImpl.canUpgradeFrom(currentImpl)` on the incoming implementation (via `IProxyAuthorization`) and then delegates the full calldata to the current implementation, whose `_authorizeUpgrade` performs caller authorization (typically `onlyOwner`). Both gates must succeed for the implementation slot to change.

Verification — `verifyContract(proxy, expectedImpl)` — reads the proxy's baked-in salt and current implementation, reconstructs the `CREATE2` address with the factory bytecode and the proxy's salt, and returns `true` only if it matches the `proxy` argument. Any revert from `getVerifiableProxyData()` is silently converted into `false` via `try/catch`. `isContract()` is implemented with `extcodesize`, which is an accepted limitation since the proxy is verified *after* its constructor has run.

4.11.1 Key Functions

- `VerifiableFactory.deployProxy(impl, salt, data)` — binds `msg.sender` into `outerSalt`, `CREATE2`-deploys `UUPSProxy`, initializes it, and — if `data.length > 0` — runs `data` against the implementation.
- `VerifiableFactory.verifyContract(proxy, expectedImpl)` — returns `true` iff `proxy` exposes the expected implementation and matches the `CREATE2` reconstruction for its stored salt.
- `UUPSProxy.initialize(impl, data)` — public but one-shot; writes the ERC-1967 implementation slot via assembly and optionally `delegatecall`s `data` to `impl`. Security depends on the factory atomically deploying and initializing in one transaction.
- `UUPSProxy.upgradeToAndCall(newImpl, data)` — enforces `newImpl.canUpgradeFrom(currentImpl)` and delegates full calldata to the current implementation, whose `_authorizeUpgrade` authorizes the caller. The fallback implementation-change guard is intentionally disabled for this path.
- `UUPSProxy.fallback() / receive()` — delegates arbitrary calls to the current implementation and reverts if the implementation slot changes during the call; accepts raw ETH.

5 Risk Rating Methodology

The risk rating methodology used by Nethermind Security follows the principles established by the OWASP Foundation. The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;

b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;

c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;

b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;

c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	**High**	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	**Medium**	**High**
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, Nethermind Security also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;

b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;

c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Medium] `UUPSProxy` upgrade validation relies on attacker-controlled `canUpgradeFrom(...)` check, allowing `WrapperRegistry` implementation replacement that bypasses frozen fuse restrictions

File(s): contracts/src/registry/WrapperRegistry.sol, contracts/src/migration/LockedWrapperReceiver.sol

Description: When a locked `.eth` name migrates from ENSv1 to ENSv2, the `LockedWrapperReceiver` contract deploys a `WrapperRegistry` proxy through the `VerifiableFactory`. The factory deploys a `UUPSProxy` instance and initializes it with the `WrapperRegistry` implementation:

```
IRegistry subregistry = IRegistry(
    VERIFIABLE_FACTORY.deployProxy(
        WRAPPER_REGISTRY_IMPL,
        uint256(node),
        abi.encodeCall(
            IWrapperRegistry.initialize,
            (
                node,
                parentRegistry,
                md.label,
                md.owner,
                _subregistryRoleBitmapFromFuses(fuses)
            )
        )
    )
);
```

The `UUPSProxy` contract provides an `upgradeToAndCall(...)` function that is meant to validate new implementations before delegating to the current one. This function calls `canUpgradeFrom(...)` on the new implementation through the `IProxyAuthorization` interface:

```
function upgradeToAndCall(
    address newImplementation,
    bytes memory
) public payable {
    // ...
    IProxyAuthorization newImpl =
        IProxyAuthorization(newImplementation);
    // @audit-issue Called on the new implementation, not a trusted source.
    if (!newImpl.canUpgradeFrom(_implementation())) {
        revert InvalidUpgradeTarget(
            _implementation(), newImplementation
        );
    }
    _delegate(_implementation(), false);
}
```

This check is called on the new implementation itself. An attacker who deploys their own contract can implement `canUpgradeFrom(...)` to return `true` for any input, rendering the validation meaningless.

After the proxy-level check passes, the call is delegated to the current `WrapperRegistry` implementation. The `_authorizeUpgrade(...)` function in `WrapperRegistry` only verifies that the caller holds `ROLE_UPGRADE` and does not inspect the `newImplementation` address:

```
function _authorizeUpgrade(
    // @audit-issue The parameter is not used.
    address
) internal override
    onlyRootRoles(RegistryRolesLib.ROLE_UPGRADE) {
    //
}
```

The `initialize(...)` function in `WrapperRegistry` unconditionally grants `ROLE_UPGRADE` and `ROLE_UPGRADE_ADMIN` to the name owner, regardless of the fuse state:

```
_grantRoles(
    ROOT_RESOURCE,
    // @audit-issue Always granted regardless of frozen fuse state.
    RegistryRolesLib.ROLE_UPGRADE
        | RegistryRolesLib.ROLE_UPGRADE_ADMIN
```

```
        | roleBitmap,
    admin,
    false
);
```

The combination of these three gaps allows the owner of a frozen name to replace the `WrapperRegistry` implementation with an arbitrary contract. A custom implementation can remove all access control checks, bypassing the frozen fuse restrictions that the `_subregistryRoleBitmapFromFuses(...)` function was designed to preserve from ENSv1.

Recommendation(s): Consider adding factory-level validation of new implementations to the upgrade path. The proxy could verify the new implementation through the `VerifiableFactory` rather than relying on a call to the attacker-controlled contract. Additionally, consider whether `_authorizeUpgrade(...)` in `WrapperRegistry` should validate the new implementation address against an expected implementation or a factory-maintained allowlist.

Status: Fixed.

Update from the client: Fixed in PR #290.

6.2 [Low] Expired name tokens can be transferred via `safeTransferFrom(...)` because `_update(...)` checks raw storage ownership instead of the expiry-aware `ownerOf(...)`

File(s): contracts/src/registry/PermissionedRegistry.sol, contracts/src/erc1155/ERC1155Singleton.sol

Description: `PermissionedRegistry` uses a two-layer ownership model. The raw storage layer (`_owners[id]`, exposed via `latestOwnerOf(...)`) retains the actual owner indefinitely. The view layer (`ownerOf(...)`) returns `address(0)` once a token expires. This separation is intentional: it allows the system to track the latest owner for premium pricing while treating expired names as available for re-registration.

The `ownerOf(...)` override in `PermissionedRegistry` returns `address(0)` when `_isExpired(entry.expiry)` is true:

```
function ownerOf(
    uint256 tokenId
) public view virtual override returns (address) {
    Entry storage entry = _entry(tokenId);
    return
        tokenId != _constructTokenId(tokenId, entry)
            || _isExpired(entry.expiry)
            // @audit-issue Returns address(0) for expired tokens.
            ? address(0)
            : super.ownerOf(tokenId);
}
```

However, the ERC1155 transfer path bypasses this expiry-aware view entirely. When `safeTransferFrom(...)` is called, two `_update(...)` functions execute in sequence, and neither checks expiry.

The `_update(...)` function in `ERC1155Singleton` reads directly from raw storage:

```
function _update(...) internal virtual {
    // ...
    for (uint256 i = 0; i < ids.length; ++i) {
        // ...
        if (value > 0) {
            // @audit-issue Reads raw _owners, not the expiry-aware ownerOf.
            address owner = _owners[id];
            if (owner != from) {
                revert ERC1155InsufficientBalance(...);
            }
            _owners[id] = to;
        }
    }
    // ...
}
```

The `_update(...)` override in `PermissionedRegistry` checks `ROLE_CAN_TRANSFER_ADMIN` via `hasRoles(...)`:

```
function _update(...) internal virtual override {
    bool externalTransfer =
        to != address(0) && from != address(0);
    if (externalTransfer) {
        for (uint256 i; i < tokenIds.length; ++i) {
            // @audit-issue Roles survive expiry. This check passes.
            if (
                !hasRoles(
                    tokenIds[i],
                    RegistryRolesLib.ROLE_CAN_TRANSFER_ADMIN,
                    from
                )
            ) {
                revert TransferDisallowed(tokenIds[i], from);
            }
        }
    }
    super._update(from, to, tokenIds, values);
    // ...
}
```

The `hasRoles(...)` function reads `_roles` storage directly, with no expiry awareness. Since roles are never revoked on expiry, this check passes for expired tokens.

This allows the owner of an expired name to transfer it to another address. The transfer moves both raw ownership and all token-level roles to the recipient. After the transfer, `latestOwnerOf(...)` returns the new owner, `ownerOf(...)` still returns `address(0)` (the token remains expired), and the new owner holds all previously assigned roles

Recommendation(s): Consider adding an expiry check in the `_update(...)` override of `PermissionedRegistry` for external transfers, reverting if the token is expired. This would align the transfer path with the view-layer semantics that treat expired tokens as non-existent.

Status: Fixed.

Update from the client: Fixed in PR #278.

6.3 [Low] Flipped operands in `gt(...)` make the non-payable check in `initialize(...)` dead code

File(s): `src/UUPSProxy.sol`

Description: The `initialize(...)` function includes a guard intended to revert when `data` is empty and `msg.value` is non-zero. This mirrors OpenZeppelin's `_checkNonPayable(...)` utility, which reverts with `ERC1967NonPayable(...)` if `msg.value > 0`.

However, the Yul comparison has its operands flipped:

```
let dlength := mload(data)
switch dlength
case 0 {
    // @audit-issue This checks 0 > callvalue(), which is always false.
    if gt(0, callvalue()) {
        mstore(0, _ERC1967_NON_PAYABLE_ERROR_SELECTOR)
        revert(0x1c, 0x04)
    }
}
```

In Yul, `gt(a, b)` returns `1` if `a > b` using unsigned 256-bit comparison. The expression `gt(0, callvalue())` evaluates to `0 > callvalue()`. Since `callvalue()` is an unsigned integer and always `>= 0`, this condition is never true. The revert branch is dead code.

The intended check is `gt(callvalue(), 0)`, which would correctly evaluate `callvalue() > 0` and revert when ETH is sent without initialization data.

Recommendation(s): Consider swapping the operand order in the `gt(...)` call so that it correctly evaluates `callvalue() > 0` instead of `0 > callvalue()`.

Status: Fixed.

Update from the client: Fixed in PR #294.

6.4 [Low] Reentrancy in `_regenerate(...)` via `ERC1155 onERC1155Received(...)` callback allows token transfer during role operations and `tokenVersionId` inflation

File(s): `contracts/src/registry/PermissionedRegistry.sol`, `contracts/src/erc1155/ERC1155Singleton.sol`

Description: `PermissionedRegistry._regenerate(...)` performs a burn-then-mint sequence to create a new `tokenId` whenever roles change on a registered name. The `_mint(...)` call triggers `ERC1155Utils.checkOnERC1155Received(...)`, which gives execution control to the token owner if it is a contract. Since `_regenerate(...)` is called from `_onRolesGranted(...)`/`_onRolesRevoked(...)` (which are themselves called in the execution of `_grantRoles(...)`/`_revokeRoles(...)`), the callback occurs while the role operation is still in progress.

The flow is:

```
grantRoles(anyId, roleBitmap, account)
  -> _grantRoles(resource, roleBitmap, account, true)
    -> _roles[resource][account] |= roleBitmap
    -> _updateRoleCounts(resource, ...)
    -> _onRolesGranted(resource, ...)
      -> _regenerate(resource)
        -> _burn(owner, oldTokenId, 1)
        -> _mint(owner, newTokenId, 1, "")
          -> onERC1155Received(owner) // <- CALLBACK
        -> emit TokenRegenerated(...)
    -> emit EACRolesChanged(...)
```

During the `onERC1155Received(...)` callback, the `TokenRegenerated` and `EACRolesChanged` events have not yet been emitted. A token-owner contract can use this callback to change the role of the token again triggering the same flow, this would cause that multiple events will be emitted in the wrong order, potentially creating issues in indexers or similar tools that use those events to keep track of token names.

Recommendation(s): Consider emitting the events before minting the new token.

Status: Fixed.

Update from the client: Fixed in PR #273.

6.5 [Low] Wildcard name roles do not authorize `setText(...)` and `setAddr(...)` due to a missing `resource(0, 0)` check in `onlyPartRoles`

File(s): contracts/src/resolver/PermissionedResolver.sol

Description: The `PermissionedResolver` contract uses a permission system built on `EnhancedAccessControl`. Permissions are tied to resources. Each resource combines a name (identified by its `node` hash) and a record type (identified by a `part` hash). The contract documents a 4-resource permission matrix in its code documentation:

```
             |       Any (*)         |      Specific (1)       |
+------------+-----------------------+-------------------------+
|    Any (*) |   resource(0, 0)      |   resource(0, <part>)   |
+------------+-----------------------+-------------------------+
| Specific   | resource(<node>, 0)   | resource(<node>, <part>)|
+------------+-----------------------+-------------------------+
```

When a setter function is called, the `onlyPartRoles` modifier checks whether the caller holds the required role in any of these resources. An admin can grant a wildcard "any name" permission by calling `grantNameRoles(...)` with an empty DNS-encoded name. This stores roles at `resource(0, 0)`, the top-left cell of the matrix.

The `onlyPartRoles` modifier only checks three of the four documented resources. It never checks `resource(0, 0)`:

```
modifier onlyPartRoles(
    bytes32 node,
    bytes32 part,
    uint256 roleBitmap
) {
    address sender = _msgSender();
    if (
        // @audit Checks resource(node, part).
        !hasRoles(
            PermissionedResolverLib.resource(node, part),
            roleBitmap,
            sender
        ) &&
        // @audit Checks resource(0, part).
        !hasRoles(
            PermissionedResolverLib.resource(0, part),
            roleBitmap,
            sender
        )
    ) {
        // @audit Checks resource(node, 0). Note that _checkRoles calls hasRoles internally.
        // @audit-issue The resource(0, 0) check is missing.
        _checkRoles(
            PermissionedResolverLib.resource(node, 0),
            roleBitmap,
            sender
        );
    }
    _;
}
```

Each `hasRoles(...)` call in `EnhancedAccessControl` implicitly checks `ROOT_RESOURCE` by combining root-level and resource-specific roles with a bitwise OR:

```
uint256 public constant ROOT_RESOURCE = 0;

function hasRoles(
    uint256 resource,
    uint256 roleBitmap,
    address account
) public view virtual returns (bool) {
    return
        (_roles[ROOT_RESOURCE][account]
            | _roles[resource][account])
            & roleBitmap == roleBitmap;
}
```

This implicit `ROOT_RESOURCE` check might seem to cover `resource(0, 0)`. It does not. `ROOT_RESOURCE` is the literal value `0`, while `resource(0, 0)` is `keccak256(abi.encode(0, 0))`, a completely different value. The `resource(...)` function in `PermissionedResolverLib` always hashes its inputs:

```
function resource(
    bytes32 node,
    bytes32 part
) internal pure returns (uint256 ret) {
    assembly {
        mstore(0, node)
        mstore(32, part)
        ret := keccak256(0, 64)
    }
}
```

The `grantNameRoles(...)` function stores roles at `resource(node, 0)`. When called with an empty name, the `node` is `bytes32(0)`, so roles are stored at `resource(0, 0)`:

```
function grantNameRoles(
    bytes calldata toName,
    uint256 roleBitmap,
    address account
) external returns (bool) {
    // @audit For empty name "", namehash returns bytes32(0).
    bytes32 node = NameCoder.namehash(toName, 0);
    uint256 resource =
        PermissionedResolverLib.resource(node, 0);
    _checkCanGrantRoles(resource, roleBitmap, _msgSender());
    emit NamedResource(resource, toName);
    return _grantRoles(resource, roleBitmap, account, true);
}
```

Only functions that pass a non-zero `part` to the modifier are affected, because for them `resource(0, part)` never equals `resource(0, 0)`. These are `setText(...)` and `setAddr(...)`:

```
function setText(
    bytes32 node,
    string calldata key,
    string calldata value
)
    external
    onlyPartRoles(
        node,
        PermissionedResolverLib.textPart(key),
        PermissionedResolverLib.ROLE_SET_TEXT
    )
{
    // ...
}
```

The same applies to `setAddr(...)`:

```
function setAddr(
    bytes32 node,
    uint256 coinType,
    bytes memory addressBytes
)
    public
    onlyPartRoles(
        node,
        PermissionedResolverLib.addrPart(coinType),
        PermissionedResolverLib.ROLE_SET_ADDR
    )
{
    // ...
}
```

When an admin calls `grantNameRoles(NameCoder.encode(""), ROLE_SET_TEXT, bob)` to give Bob permission to set text records on all names, Bob's roles are stored at `resource(0, 0)`. When Bob then calls `setText(someNode, "avatar", "...")`, the modifier checks `resource(someNode, textPart("avatar"))`, `resource(0, textPart("avatar"))`, and `resource(someNode, 0)`. None of these match `resource(0, 0)` where Bob's roles are stored. The transaction reverts.

Functions that use `part = 0` (such as `setContenthash(...)`, `setName(...)`, `setPubkey(...)`, `setABI(...)`, and `setInterface(...)`) are not affected. When `part` is `0`, the second check in the modifier becomes `hasRoles(resource(0, 0), ...)`, which matches correctly.

The code documentation on `grantNameRoles(...)` also claims that calling it with an empty name "is equivalent to `grantRootRoles(...)`". This is incorrect.`grantRootRoles(...)` stores roles at `ROOT_RESOURCE` (literal `0`), which `hasRoles(...)` always checks implicitly. `grantNameRoles(...)` with an empty name stores at `resource(0, 0)` (`keccak256(abi.encode(0, 0))`), which is not checked implicitly.

The practical impact is limited. No unauthorized access is possible. The bug causes a denial of expected functionality for `setText(...)` and `setAddr(...)` when roles are granted through `grantNameRoles(...)` with an empty name. An admin can achieve the intended wildcard authorization by using `grantRootRoles(...)` instead.

Recommendation(s): Consider updating the `onlyPartRoles` modifier to cover all four cells of the documented permission matrix. Consider also revising the code documentation on `grantNameRoles(...)` to accurately reflect its behavior when called with an empty name.

Status: Fixed.

Update from the client: Fixed in PR #279.

6.6 [Info] Name-level admins can grant key-specific text and addr roles but cannot revoke them

File(s): contracts/src/resolver/PermissionedResolver.sol, contracts/src/access-control/EnhancedAccessControl.sol

Description: The PermissionedResolver contract allows name-level admins to delegate fine-grained write access for a specific text key or coin type. A name admin holding ROLE_SET_TEXT_ADMIN at resource(node, 0) can call grantTextRoles(...) to give an account permission to write a single text key. The same applies to grantAddrRoles(...) for a single coin type.

These two functions check authorization at the name-level resource but store the granted role at a more specific resource.
In grantTextRoles(...), the authorization check targets resource(node, 0) while the role is stored at resource(node, textPart(key)):

```
function grantTextRoles(
    bytes calldata toName,
    string calldata key,
    address account
) external returns (bool) {
    bytes32 node = NameCoder.namehash(toName, 0);
    _checkCanGrantRoles( // @audit Authorization checked at resource(node, 0).
        PermissionedResolverLib.resource(node, 0),
        PermissionedResolverLib.ROLE_SET_TEXT,
        _msgSender()
    );
    uint256 resource = PermissionedResolverLib.resource(
        node,
        PermissionedResolverLib.textPart(key)
    );
    return _grantRoles( // @audit-issue Role stored at resource(node, textPart(key)).
        resource,
        PermissionedResolverLib.ROLE_SET_TEXT,
        account,
        true
    );
}
```

The grantAddrRoles(...) function follows the same pattern, storing ROLE_SET_ADDR at resource(node, addrPart(coinType)).

To revoke these roles, the only available function is revokeRoles(...) inherited from EnhancedAccessControl. This function is not overridden in PermissionedResolver. Its authorization check uses _getRevokableRoles(...), which only looks for admin roles at the exact resource being revoked and at ROOT_RESOURCE:

```
function _getRevokableRoles(
    uint256 resource,
    address account
) internal view virtual returns (uint256) {
    // @audit-issue Only checks the target resource and ROOT_RESOURCE.
    uint256 adminRoleBitmap =
        (_roles[resource][account]
            | _roles[ROOT_RESOURCE][account])
            & EACBaseRolesLib.ADMIN_ROLES;
    uint256 regularRoles = adminRoleBitmap >> 128;
    return regularRoles | adminRoleBitmap;
}
```

When a name admin calls revokeRoles(resource(node, textPart(key)), ROLE_SET_TEXT, account), the check looks for admin roles at resource(node, textPart(key)). The name admin only holds admin at resource(node, 0), which is a different resource. The call reverts.

No function in the PermissionedResolver API grants admin roles at key-specific resources. The grantTextRoles(...) function only grants ROLE_SET_TEXT, not its admin counterpart. The grantNameRoles(...) function only stores roles at resource(node, 0). The grantRoles(...) function is overridden to always revert.

This creates an asymmetry: the grant path and the revoke path check authorization at different resources. A name admin can delegate key-specific permissions but cannot revoke them. Only ROOT_RESOURCE admin can revoke these roles.

The practical impact is limited. The irrevocable permission only covers a single text key or coin type on a single name. The name admin retains write access at the broader resource(node, 0) and can always overwrite data set by the grantee. The ROOT_RESOURCE admin can revoke the roles if needed.

Recommendation(s): Consider providing a revocation path that allows name-level admins to undo key-specific delegations without requiring ROOT_RESOURCE admin intervention. Alternatively, consider documenting this asymmetry so that name admins are aware of the limitation.

Status: Fixed.

Update from the client: Fixed in PR #279.

6.7 [Info] Out-of-bounds memory write in `ResolverProfileRewriterLib.replaceNode(...)` due to unvalidated ABI offset pointers can return incorrect resolution data

File(s): contracts/src/resolver/libraries/ResolverProfileRewriterLib.sol, contracts/src/resolver/PermissionedResolver.sol

Description: `ResolverProfileRewriterLib.replaceNode(...)` recursively traverses ABI-encoded `multicall(bytes[])` calldata to replace the `bytes32 node` parameter in each nested resolver call. The traversal is implemented in Yul assembly and follows offset pointers embedded in the user-supplied `fromData` without validating that they remain within the bounds of the memory-copied buffer.

An attacker can craft `fromData` containing a `multicall(...)` with element offsets that, via `uint256` wraparound arithmetic, point outside the allocated buffer to arbitrary memory addresses. The `default` branch of the `replace(...)` function then writes the computed namehash (`node`) at `new_ptr + 36`, which can target critical memory regions such as the free memory pointer at `0x40` or the Solidity zero slot at `0x60`.

The `replace(...)` Yul function processes multicall offsets as follows:

```
off := add(ptr, 36)
off := add(off, mload(off))
size := shl(5, mload(off))
for { } size { size := sub(size, 32) } {
    // @audit-issue Element offset is read from user data
    // and used in pointer arithmetic with no bounds check.
    replace(
        add(
            add(off, 32),
            mload(add(off, size))
        ),
        node
    )
}
```

`mload(add(off, size))` reads the element offset from user data. This value is used directly in pointer arithmetic with no bounds check. By setting an element offset to `2^256 - off - 32 + TARGET`, the attacker exploits `uint256` wraparound to redirect the write to any chosen `TARGET` address.

The vulnerable `replaceNode(...)` is called by `PermissionedResolver.resolve(...)`, the entry point for ENSIP-10 / CCIP-read resolution. After `replaceNode(...)` returns with corrupted memory, `resolve(...)` proceeds to `abi.decode(...)` the result and issue `staticcall`s to itself with the decoded data. Because the free memory pointer is corrupted, `abi.decode(...)` allocates at an unexpected address, reading uninitialized or attacker-influenced memory. This can cause the function to return incorrect resolution data or revert.

Since `resolve(...)` is a `view` function, no on-chain state changes result. However, any on-chain contract that calls `resolve(...)` with partially attacker-influenced multicall content and acts on the return value (e.g., resolving an address to send funds) could receive incorrect data.

Recommendation(s): Add a bounds check to the `replace(...)` function that validates each element offset stays within the allocated buffer. Pass the buffer length alongside the pointer and revert if any computed `new_ptr` falls outside `[copy, copy + length]`.

Status: Fixed.

Update from the client: Fixed in commit 567735.

6.8 [Info] The proxy `receive(...)` function prevents delegation of plain ETH transfers to the implementation

File(s): src/UUPSProxy.sol

Description: The `UUPSProxy` contract uses a `fallback(...)` function to delegate all incoming calls to the implementation contract. This delegation mechanism forwards the full calldata to the implementation via `delegatecall`. This allows the proxy to support any function the implementation exposes.

The proxy also defines a `receive(...)` function with an empty body:

```
fallback() external payable {
    _delegate(_implementation(), true);
}

// @audit-issue The empty receive absorbs plain ETH transfers.
receive() external payable {}
```

In Solidity, when a contract receives a call with empty calldata and `msg.value != 0`, the EVM checks for a `receive(...)` function first. Only if no `receive(...)` function exists does the call fall through to `fallback(...)`. Because the proxy defines its own `receive(...)`, plain ETH transfers are handled entirely by the proxy. The `fallback(...)` function is never reached, and no `delegatecall` to the implementation occurs.

This means that any `receive(...)` logic defined in the implementation contract is unreachable for plain ETH transfers. If the implementation relies on `receive(...)` for accounting, emitting events, or ETH routing, that logic will silently be skipped. The ETH is accepted and held by the proxy, but the implementation's storage is never updated to reflect the deposit.

Recommendation(s): Consider removing the `receive(...)` function from the `UUPSProxy` contract so that plain ETH transfers fall through to `fallback(...)` and are delegated to the implementation. If implementations are not expected to handle plain ETH transfers, that restriction should be enforced in the implementation itself rather than silently in the proxy.

Status: Fixed.

Update from the client: Fixed in PR #294.

6.9 [Info] `setName(...)` and `setNameForAddr(...)` do not advance `inceptionOf`, leaving prior signatures valid after a direct name change

File(s): contracts/src/reverse-registrar/L2ReverseRegistrar.sol

Description: `L2ReverseRegistrar` provides two ways to set a reverse name. The direct path uses `setName(...)` and `setNameForAddr(...)`, where the caller (or an authorized party) submits a transaction. The signature path uses `setNameForAddrWithSignature(...)` and `setNameForOwnableWithSignature(...)`, where a relayer submits a signed message on behalf of the user.

The signature path includes replay protection through `_validateAndUpdateInception(...)`. This function requires `claim.signedAt > inceptionOf[addr]` and advances `inceptionOf[addr]` to `claim.signedAt` on success. This prevents not only replaying the same signature, but also invalidates any generated signature with a lower `signedAt` value:

```
function _validateAndUpdateInception(
    address addr,
    uint256 signedAt
) internal {
    uint256 currentInception = inceptionOf[addr];
    if (signedAt <= currentInception)
        revert StaleSignature(signedAt, currentInception);
    if (signedAt > block.timestamp)
        revert SignatureNotValidYet(
            signedAt, block.timestamp
        );
    // @audit-issue Only the signature path updates this value.
    inceptionOf[addr] = signedAt;
}
```

The direct path does not call `_validateAndUpdateInception(...)`:

```
function setName(string calldata name) external {
    // @audit-issue inceptionOf[msg.sender] is not updated.
    _setName(msg.sender, name);
}

function setNameForAddr(
    address addr,
    string calldata name
) external authorized(addr) {
    // @audit-issue inceptionOf[addr] is not updated.
    _setName(addr, name);
}
```

Because the direct path does not advance `inceptionOf`, any signature with `signedAt` greater than the current inception remains valid after a direct name change. If a user sets their name via `setName(...)`, a previously captured signature can still be submitted through the signature path to revert the name.

Recommendation(s): Consider advancing `inceptionOf[addr]` to `block.timestamp` in both `setName(...)` and `setNameForAddr(...)` so that direct name changes invalidate all prior signatures.

Status: Fixed.

Update from the client: Fixed in PR #289.

6.10 [Best Practices] Unused code

File(s): contracts/src/erc1155/ERC1155Singleton.sol, contracts/src/registry/SimpleRegistryMetadata.sol, contracts/src/registry/BaseUriRegistryMetadata.sol, contracts/src/reverse-registrar/L2ReverseRegistrar.sol, contracts/src/universalResolver/libraries/LibRegistry.sol, contracts/src/access-control/EnhancedAccessControl.sol

Description: Several contracts contain unused declarations, including events, errors, constants, and internal functions. The following unused declarations were identified:

In `ERC1155Singleton`:

- `event Approval(address,address,uint256)`: declared but never emitted. Only `ApprovalForAll` is used;
- `_mintBatch(...)` and `_burnBatch(...)`: internal functions that are never called by any inheriting contract;

In `SimpleRegistryMetadata` and `BaseUriRegistryMetadata`:

- `_ROLE_UPDATE_METADATA_ADMIN`: constant declared in both contracts but never referenced. Only the base `_ROLE_UPDATE_METADATA` constant is used;

In `L2ReverseRegistrar`:

- `error ChainIdsNotAscending()` and `error CurrentChainNotFound(uint256)`: declared but never reverted. The `ChainIdsBuilderLib` library contains its own copies of these errors and is the only place where they are actually used;

In `LibRegistry`:

- `findParentRegistry(...)`: internal function with no callers in production code;

In `EnhancedAccessControl`:

- `_revokeAllRoles(...)`: internal function with no callers in production code;

Recommendation(s): Consider removing all unused declarations to keep the codebase clean, readable, and easier to maintain. For internal functions that are only used in tests, consider moving them into test-specific helper contracts instead.

Status: Fixed.

Update from the client: Fixed in PR #280.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about ENS's contracts documentation

The ENS team provided an overview of the system components during the kick-off call with a detailed explanation of the intended functionalities and use cases of the system. Moreover, the team was responsive on Telegram and provided comprehensive documentation of the architecture and audit scope of the project, including a detailed Audit README covering the audit scope, threat model, and system invariants, a high-level ENS V2 Overview describing the architecture and design rationale for the new system, and thorough pre-migration documentation outlining the step-by-step process for migrating from ENS V1 to V2. Additionally, the codebase contained detailed NatSpec comments throughout the contracts, which made the review process clear and efficient.

8 Test Suite Evaluation

The ENS V2 protocol's test suite is comprehensive and clearly defines the expected contract behavior. To assess the depth of test coverage, the Nethermind Security team applied a technique called mutation testing to uncover untested paths. This technique introduces slight modifications to the code called "mutations" or "mutants." An example of a mutation is changing the operator in an expression `(a + b)` to `(a - b)`, or removing a `require(a > b)` statement entirely from the code. With these changes, the code no longer follows the expected business logic, and the test suite should reflect that by failing.

Evaluation of the test suite with mutation testing consists of two phases:

a. Generating modified versions of each contract, called "mutants."

b. Inserting each mutant into the original codebase and running the test suite.

Only one modification can be tested at a time. If a contract has ten mutations, the test suite must run ten times (once for every mutation). If any test fails, the mutant is considered "slain" or "killed." If no test fails, the mutant "survived," indicating a gap in test coverage that a new test case could close.

Not all surviving mutants represent real test gaps. Some mutations produce code that behaves identically to the original for all possible inputs. These are called **equivalent mutants**. For example, mutating `x * 1` to `x / 1` produces the same result for all inputs; no test can ever distinguish them. We identified approximately 140 such equivalent mutants among the survivors (33% of all survivors) and excluded them from the actionable findings below. After deduplication of overlapping gaps, **43 actionable test gaps** remain.

The following table outlines the results of the analysis performed on the ENS V2 smart contracts. The first column lists the contracts that were tested using mutation testing. The second column indicates the number of mutants generated and how many were "slain" (i.e., caught by the test suite). The third column provides the percentage of mutants slain, reflecting the effectiveness of the test suite in covering the particular contract. The higher the score, the better the test suite is at finding bugs.

Contract	Mutants (slain / total)	Score
EnhancedAccessControl.sol	238 / 247	96.36%
DNSAliasResolver.sol	1 / 12	8.33%
DNSTLDResolver.sol	14 / 103	13.59%
DNSTXTResolver.sol	2 / 50	4.00%
DNSTXTParserLib.sol	123 / 148	83.11%
ERC1155Singleton.sol	34 / 67	50.75%
HCAEquivalence.sol	5 / 5	100.00%
AbstractWrapperReceiver.sol	32 / 34	94.12%
LockedMigrationController.sol	1 / 1	100.00%
LockedWrapperReceiver.sol	102 / 107	95.33%
UnlockedMigrationController.sol	20 / 23	86.96%
LibMigration.sol	1 / 3	33.33%
BatchRegistrar.sol	12 / 14	85.71%
ETHRegistrar.sol	87 / 110	79.09%
StandardRentPriceOracle.sol	176 / 188	93.62%
LibHalving.sol	475 / 486	97.74%
BaseUriRegistryMetadata.sol	4 / 14	28.57%
MetadataMixin.sol	3 / 3	100.00%
PermissionedRegistry.sol	127 / 131	96.95%
SimpleRegistryMetadata.sol	4 / 14	28.57%
UserRegistry.sol	2 / 4	50.00%
WrapperRegistry.sol	17 / 20	85.00%
AbstractMirrorResolver.sol	0 / 2	0.00%
ENSV1Resolver.sol	2 / 2	100.00%
ENSV2Resolver.sol	3 / 6	50.00%
PermissionedResolver.sol	107 / 119	89.92%
PermissionedResolverLib.sol	26 / 120	21.67%
ResolverProfileRewriterLib.sol	1 / 1	100.00%
L2ReverseRegistrar.sol	39 / 42	92.86%
L2ReverseRegistrarWithMigration.sol	6 / 6	100.00%
StandaloneReverseRegistrar.sol	19 / 19	100.00%
ChainIdsBuilderLib.sol	1 / 2	50.00%
UniversalResolverV2.sol	2 / 2	100.00%

Contract	Mutants (slain / total)	Score
UpgradableUniversalResolverProxy.sol	25 / 26	96.15%
LibRegistry.sol	62 / 64	96.88%
LibLabel.sol	11 / 12	91.67%
Total	**1784 / 2207**	**80.83%**

Remarks about the ENS V2 Test Suite

The test suite for the ENS V2 protocol achieves an overall mutation score of 80.83%. Core contracts central to the V2 upgrade, `EnhancedAccessControl` (96.36%), `PermissionedRegistry` (96.95%), and `PermissionedResolver` (89.92%), demonstrate strong coverage, and the access control, registration, and resolver write paths are well exercised.

A notable pattern revealed by mutation testing is the presence of **unused code paths** that inflate the survivor count. Batch functions in `ERC1155Singleton` (`_mintBatch`, `_burnBatch`), the `_revokeAllRoles` internal function in `EnhancedAccessControl`, `findParentRegistry` in `LibRegistry`, and several role constants in the metadata contracts are declared but never called in production code. These account for a significant portion of surviving mutants and were independently flagged as a Best Practices finding during the manual code review.

The **DNS module** has the lowest mutation scores in the codebase: `DNSTXTResolver` (4.00%), `DNSAliasResolver` (8.33%), and `DNSTLDResolver` (13.59%). These contracts lack dedicated unit tests and are only partially covered by integration tests. The surviving mutants span TXT record parsing, hex validation, selector dispatch, and the bracket-depth tracking state machine in `DNSTXTParserLib`.

In `ERC1155Singleton` (50.75%), the ERC1155 approval and transfer-on-behalf paths are entirely untested—no test has a third party call `safeTransferFrom` via an approved operator. This gap is relevant to the Low severity finding on expired token transfers, where the `_update` function reads raw storage ownership without checking expiry. The singleton enforcement check (`value > 1`) is similarly unexercised.

In the **migration layer**, the fuse-to-role translation in `LockedWrapperReceiver` is only tested with minimal fuse configurations. Tests never migrate a locked name with `CANNOT_CREATE_SUBDOMAIN` or `CANNOT_TRANSFER` fuses set. This gap is adjacent to the Medium severity finding where the unconditional `ROLE_UPGRADE` grant in `WrapperRegistry.initialize()` allows frozen-fuse restrictions to be bypassed through a proxy upgrade.

Low-score files such as `PermissionedResolverLib` (21.67%), `SimpleRegistryMetadata` (28.57%), and `BaseUriRegistryMetadata` (28.57%) are largely explained by equivalent mutants in role constant definitions, where shifting a constant by 128 bits and mutating that shift produces values that tests cannot distinguish because they reference the same constant by name.

The following is a list of code modifications not caught by the existing test suite. Each point highlights a scenario that could benefit from higher test coverage. Survived mutants that test the same gap are grouped together, and equivalent mutants (which produce identical behavior to the original code) are excluded.

8.1 Access Control & Role Constants

PermissionedResolverLib.sol, SimpleRegistryMetadata.sol, BaseUriRegistryMetadata.sol — *~54 killable mutants out of 114 total (rest are equivalent)*

- Role constants such as `ROLE_SET_ADDR_ADMIN` are defined as `ROLE_SET_ADDR << 128`. Tests reference these constants by name but never assert their numeric values. If a constant had a copy-paste error in the shift amount, both the production code and the test would use the same wrong value, and no test would detect it. For example, mutating `ROLE_SET_ADDR << 128` to `128 << ROLE_SET_ADDR` does not cause any test to fail.

EnhancedAccessControl.sol — *3 mutants*

- In `_grantRoles()` and `_revokeRoles()`, the internal call to `_checkRoleBitmap()` that rejects malformed bitmaps is never exercised because the external `grantRoles()` modifier rejects bad bitmaps first. Replacing `_checkRoleBitmap(roleBitmap)` with `assert(true)` does not cause any test to fail. Consider adding a test that calls the internal `_grantRoles()` directly with an invalid bitmap.
- When granting roles that an account already partially holds, the assignee counter update via `_updateRoleCounts(resource, newlyAddedRoles, true)` can be replaced with `assert(true)` without any test failing. Consider adding a test that grants overlapping roles to the same account (e.g., grant role A, then grant roles A+B) and asserts the exact assignee count.

8.2 Core Registry & ERC1155

ERC1155Singleton.sol — *32 mutants*

- No test has a third party call `safeTransferFrom` on behalf of a token owner. Hardcoding the approval check `from != sender && !isApprovedForAll(from, sender)` to `false` does not cause any test to fail. The approval revocation path (`setApprovalForAll(addr, false)`) is similarly unexercised.
- `balanceOfBatch()` is never called in any test. Length validation, the loop, and return values are all uncovered (8 mutants).
- No test attempts to mint or transfer with `value > 1` and asserts the revert. All operations use `value = 1`. If the singleton enforcement were bypassed, multiple owners could exist for the same token (2 mutants).
- The batch functions (`_mintBatch`, `_burnBatch`, `_safeBatchTransferFrom`) are inherited from OpenZeppelin's ERC1155 but never called by the registry. All 18 mutants in these functions survive. These are standard inherited functions with no current code path to trigger them.
- No test calls `setApprovalForAll(address(0), true)` and asserts it reverts. The zero-address guard is unexercised (2 mutants).

PermissionedRegistry.sol — *3 mutants*

- When a name expires and a new owner re-registers it, the `_burn(prevOwner, tokenId, 1)` call can be replaced with `assert(true)` without any test failing. No test checks that the old owner's `balanceOf` becomes 0 after re-registration.
- The `sender == address(this)` bypass that allows `WrapperRegistry._inject()` to promote a RESERVED name to REGISTERED is only covered by integration tests. Hardcoding `sender != address(this)` to `false` does not cause any PermissionedRegistry unit test to fail.
- Passing `expiry = 0` to preserve the current expiry on promotion is untested. Hardcoding the `expiry == 0` condition to `false` does not cause any test to fail.

8.3 DNS Module

DNSTLDResolver.sol — *66 mutants*

- In `getDNSSECRecordsCallback()`, the two-pass TXT record iteration loop is entirely uncovered by unit tests. Mutations to the loop counter, the `RRUtils.next()` iterator advance, and the array allocation all survive (14 mutants).
- In `_parseTXTRecord()`, substring boundary arithmetic (`sep + 1`, `n - sep - 1`, `sep - p`) is untested for edge cases: separator at end of string, single-character resolver, empty context, or no separator found. Mutating `sep + 1` to `sep - 1` does not cause any test to fail (40 mutants).
- In `_parseResolver()` and `_readTXT()`, hex string validation is untested with invalid inputs: non-hex characters, wrong-length strings, and missing `0x` prefix (12 mutants).

DNSTXTResolver.sol — *49 mutants*

- The `resolve()` function dispatches to different record types based on function selectors. No unit tests exist; all selector comparisons and type-specific encoding are only covered by integration tests. Hardcoding `selector == IContentHashResolver.contenthash.selector` to `true` does not cause any test to fail (25 mutants).
- The multicall path calls itself via `staticcall` in a loop. No test verifies that when an inner call fails, the error data is correctly forwarded (7 mutants).
- When resolving a non-ETH EVM coin type, the fallback to the `a[e0]=` default record is untested. Hardcoding `useDefault && v.length == 0` to `true` does not cause any test to fail (8 mutants).
- Hex validation in `_parse0xString()` is untested with invalid inputs: non-hex characters, missing `0x` prefix, and strings shorter than 2 characters (9 mutants).

DNSTXTParserLib.sol — *25 mutants*

- The bracket-depth tracking in `find()` handles keys like `a[e60]=`. No test uses unbalanced brackets, nested brackets at depth > 1, or keys with `[` but no matching `]`. Replacing `++depth` with `assert(true)` (disabling bracket tracking entirely) does not cause any test to fail. A crafted TXT record with unbalanced brackets could cause the parser to misidentify key boundaries (19 mutants).
- Whitespace handling in the `find()` state machine is untested with leading/trailing whitespace and multi-space separators (6 mutants).

DNSAliasResolver.sol — *9 mutants*

- No unit tests exist for `DNSAliasResolver`. Suffix matching, offset arithmetic for name rewriting, and the non-matching-suffix revert path are only covered by integration tests. Mutating `sep + 1` to `sep - 1` in the suffix rewrite does not cause any test to fail.

ENSV2Resolver.sol — *3 mutants*

- No unit tests exist for `ENSV2Resolver`. The special case where `.eth` names use a v1 resolver during migration is completely untested. If the override were skipped, `.eth` names would resolve via the v2 registry instead of the intended v1 resolver during the migration period.

AbstractMirrorResolver.sol — *1 mutant*

- Replacing the entire `callResolver(...)` invocation in `resolve()` with `assert(true)` does not cause any test to fail. The function relies on CCIP-read (revert-based response pattern) and has no unit test coverage.

8.4 Registrar & Pricing

ETHRegistrar.sol / **StandardRentPriceOracle.sol** — *13 mutants*

- Premium pricing is tested in isolation via `LibHalving` unit tests, but no test registers a name as a new (non-prior) owner during the premium decay window through the full `ETHRegistrar.register()` flow. The test setup warps past `premiumPeriod`, making premium = 0 for all registrations. Hardcoding the premium trigger `owner != address(0) && owner != state.latestOwner` to `false` does not cause any test to fail (10 mutants).
- In `updatePremiumPricing()`, the test uses `halvingPeriod = 1`, so a mutation replacing the parameter with the literal `1` is invisible. Using a different test value (e.g., `halvingPeriod = 86400`) would exercise the assignment (1 mutant).
- In `updatePaymentToken()`, no test updates an already-active payment token's ratio or removes an inactive token to verify event emission edge cases (2 mutants).

LibHalving.sol — *4 actionable mutants (7 additional are equivalent)*

- No test calls `halving()` with `halvingPeriod = 0` (which should return 0, not revert). No test uses fractional exponents beyond exact powers-of-2, so the fractional interpolation math is not exercised. The `halvingPeriod = 0` guard prevents division-by-zero that would cause all `rentPrice` queries to revert.

BatchRegistrar.sol — *2 mutants*

- No test passes mismatched `labels.length != expires.length` to verify the `require` check. Hardcoding the condition to `true` does not cause any test to fail.

8.5 Resolver

UserRegistry.sol, WrapperRegistry.sol, PermissionedResolver.sol — *4 mutants*

- No test calls `initialize()` directly on the implementation contract (not the proxy) to assert it reverts. Without `_disableInitializers()`, someone could initialize the implementation and set themselves as admin. Replacing `_disableInitializers()` with `assert(true)` does not cause any test to fail.

PermissionedResolver.sol — *7 mutants*

- In `ABI()`, the content type bitmask filter `contentType & contentTypes` can be mutated to `contentType + contentTypes` without any test failing. Tests always query with `contentTypes = type(uint256).max`, so any operator produces the same result (2 mutants).
- In `interfaceImplementer()`, only the positive ERC165 case is tested. No test checks that a contract that does NOT support the interface correctly returns `address(0)`. Hardcoding the `supportsInterface` check to `false` does not cause any test to fail (2 mutants).
- In `multicall()` and `resolve()`, write-path tests only check side effects, not return values. Replacing `results[i] = v` with `assert(true)` does not cause any test to fail (3 mutants).

8.6 Migration

LockedWrapperReceiver.sol — *5 mutants*

- In `_tokenRoleBitmapFromFuses()`, the `CANNOT_CREATE_SUBDOMAIN` and `CANNOT_TRANSFER` fuse checks are always false in tests. No test migrates a locked name with these fuses set and verifies that the corresponding roles are correctly withheld (2 mutants).
- Role bitmaps combined with bitwise OR (`|`) are only tested with non-overlapping bits. Mutating `|` to `+` in `RegistryRolesLib.ROLE_UPGRADE_ADMIN | roleBitmap` does not cause any test to fail. With overlapping bits, `+` would produce a corrupted bitmap granting unintended roles (3 mutants across `LockedWrapperReceiver` and `WrapperRegistry`).

UnlockedMigrationController.sol — *3 mutants*

- In `onERC721Received()`, no test calls from a non-`BaseRegistrar` address to verify the sender check reverts. Hardcoding `msg.sender != address(_REGISTRAR_V1)` to `false` does not cause any test to fail. Without this check, anyone could call the migration callback directly to claim premigrated name reservations without owning the v1 token.
- In `onERC721Received()`, no test transfers a `BaseRegistrar` token while providing a label whose hash does not match the `tokenId`. Hardcoding the mismatch check to `true` does not cause any test to fail.
- In `onERC1155Received()`, after wrapped unlocked migration, no test asserts the v1 resolver is cleared to `address(0)`. Replacing `NAME_WRAPPER.setResolver(bytes32(id), address(0))` with `assert(true)` does not cause any test to fail.

AbstractWrapperReceiver.sol / **LibMigration.sol** — *4 mutants*

- The batch data validation uses `ids.length * MIN_DATA_SIZE` where `MIN_DATA_SIZE = 7 * 32 = 224`. Tests use small batches where mutations to `*` (e.g., to `/`) still produce passing values. Mutating `7 * 32` to `7 + 32` in the constant does not cause any test to fail.

8.7 Miscellaneous

L2ReverseRegistrar.sol — *2 mutants*

- In `setNameForAddrWithSignature()`, all ERC6492 tests use valid signatures. No test submits an invalid ERC6492-formatted signature and verifies rejection. Hardcoding the ERC6492 detection suffix check to `true` does not cause any test to fail. Without the `isValidSig` check, anyone could claim any address's reverse name by appending the ERC6492 suffix to arbitrary data.

LibRegistry.sol — *2 mutants*

- In `findCanonicalName()`, no test creates a registry whose `getSubregistry(label)` returns itself. The self-referencing guard that prevents infinite loops is never exercised.
- In `findParentRegistry()`, no test calls with the root name (empty label hash) to verify the boundary case.

UpgradableUniversalResolverProxy.sol — *1 mutant*

- The constructor calls `_validateImplementation()` to verify the implementation is valid. No test deploys the proxy with an invalid implementation. The `upgradeTo` path IS tested, but the constructor path is not.

PermissionedResolver.sol — *1 mutant*

- In `setAddr()`, the `AddrChanged` legacy event is emitted only when `coinType == COIN_TYPE_ETH`. No test verifies that this event is NOT emitted for other coin types. If always emitted, non-ETH address changes would produce spurious events with truncated addresses, confusing off-chain indexers.

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to Nethermind in order that Nethermind could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. Nethermind has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.